

第 10 章 数学概念与方法

学习目标

- ☑ 熟练掌握扩展欧几里德算法和它的时间复杂度
- ☑ 熟练掌握用筛法构造素数表，了解素数定理
- ☑ 学会求二元线性不定方程的整数解
- ☑ 熟练掌握模运算规则、快速幂取模算法和模线性方程的解法
- ☑ 熟悉杨辉三角、二项式定理和组合数的基本性质
- ☑ 学会推导约数个数公式和欧拉函数公式
- ☑ 熟练掌握可重集全排列的编码和解码算法
- ☑ 理解样本空间、事件和概率，学会用组合计数的方法计算离散概率
- ☑ 理解条件概率的概念和计算方法
- ☑ 理解连续概率和数学期望的概念和计算方法
- ☑ 熟悉常见计数序列，如 Fibonacci 数列、Catalan 数列等
- ☑ 熟悉建立递推关系的基本方法、常见错误和实现技巧

没有数学就没有算法；没有好的数学基础，也很难在算法上有所成就。本章介绍算法竞赛中涉及的常见数学概念和方法，包括数论、排列组合、递推关系和离散概率等。

10.1 数论初步

数论被“数学王子”高斯誉为整个数学王国的皇后。在算法竞赛中，数论常常以各种面貌出现，但万变不离其宗，大部分数论题目并不涉及多少特殊的知识，但对数学思维和能力要求较高。本节介绍几个最为常用的算法，并通过例题展示一些常用的思维方式。

10.1.1 欧几里德算法和唯一分解定理

除法表达式。给出一个这样的除法表达式： $X_1 / X_2 / X_3 / \cdots / X_k$ ，其中 X_i 是正整数。除法表达式应当按照从左到右的顺序求和，例如，表达式 $1/2/1/2$ 的值为 $1/4$ 。但可以在表达式中嵌入括号以改变计算顺序，例如，表达式 $(1/2)/(1/2)$ 的值为 1 。

输入 $X_1, X_2, \dots, X_k$ ，判断是否可以通过添加括号，使表达式的值为整数。 $K \leq 10000$ ， $X_i \leq 10^9$ 。

【分析】

表达式的值一定可以写成 A/B 的形式： A 是其中一些 X_i 的乘积，而 B 是其他数的乘积。

不难发现, X_2 必须放在分母位置, 那其他数呢?

幸运的是, 其他数都可以在分子位置:

$$E = X_1 / (X_2 / X_3 / \cdots X_k) = \frac{X_1 X_3 X_4 \cdots X_k}{X_2}$$

接下来的问题就变成了: 判断 E 是否为整数。

第1种方法是利用前面介绍的高精度运算: k 次乘法加一次除法。显然, 这个方法是正确的, 但却比较麻烦。

第2种方法是利用唯一分解定理, 把 X_2 写成若干素数相乘的形式:

$$X_2 = p_1^{a_1} p_2^{a_2} p_3^{a_3} \cdots$$

然后依次判断每个 $p_i^{a_i}$ 是否是 $X_1 X_3 X_4 \cdots X_k$ 的约数。这次不用高精度乘法了, 只需把所有 X_i 中 p_i 的指数加起来。如果结果比 a_i 小, 说明还会有 p_i 约不掉, 因此 E 不是整数。这种方法在第5章中已经用过, 这里不再赘述。

第3种方法是直接约分: 每次约掉 X_i 和 X_2 的最大公约数 $\gcd(X_i, X_2)$, 则当且仅当约分结束后 $X_2=1$ 时 E 为整数, 程序如下:

```
int judge(int* X) {
    X[2] /= gcd(X[2], X[1]);
    for(int i = 3; i <= k; i++) X[2] /= gcd(X[i], X[2]);
    return X[2] == 1;
}
```

整个算法的时间效率取决于这里的 $\gcd$ 算法。尽管依次试除也能得到正确的结果, 但还有一个简单、高效, 而且相当优美的算法——辗转相除法。它也许是最广为人知的数论算法。

辗转相除法的关键在于如下恒等式: $\gcd(a, b) = \gcd(b, a \bmod b)$ 。它和边界条件 $\gcd(a, 0)=a$ 一起构成了下面的程序:

```
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

这个算法称为欧几里德算法 (Euclid algorithm)。既然是递归, 那么免不了问一句: 会栈溢出吗? 答案是不会。可以证明, $\gcd$ 函数的递归层数不超过 $4.785 \lg N + 1.6723$, 其中 $N = \max\{a, b\}$ 。值得一提的是, 让 $\gcd$ 递归层数最多的是 $\gcd(F_n, F_{n-1})$, 其中 F_n 是后文要介绍的 Fibonacci 数。

利用 $\gcd$ 还可以求出两个整数 a 和 b 的最小公倍数 $\text{lcm}(a, b)$ 。这个结论很容易由唯一分解定理得到。设

$$\begin{aligned} a &= p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r} \\ b &= p_1^{f_1} p_2^{f_2} \cdots p_r^{f_r} \end{aligned}$$

则

$$\begin{aligned} \gcd(a, b) &= p_1^{\min\{e_1, f_1\}} p_2^{\min\{e_2, f_2\}} \cdots p_r^{\min\{e_r, f_r\}} \\ \text{lcm}(a, b) &= p_1^{\max\{e_1, f_1\}} p_2^{\max\{e_2, f_2\}} \cdots p_r^{\max\{e_r, f_r\}} \end{aligned}$$

由此不难验证 $\gcd(a,b) * \text{lcm}(a,b) = a * b$ 。不过即使有了公式也不要大意。如果把 lcm 写成 $a * b / \gcd(a,b)$ ，可能会因此丢掉不少分数—— $a * b$ 可能会溢出！正确的写法是先除后乘，即 $a / \gcd(a,b) * b$ 。这样一来，只要题面上保证最终结果在 int 范围之内，这个函数就不会出错。但前一份代码却不是这样：即使最终答案在 int 范围之内，也有可能中间过程越界。注意这样的细节，毕竟算法竞赛不是数学竞赛。

10.1.2 Eratosthenes 筛法

无平方因子的数。给出正整数 n 和 m ，区间 $[n, m]$ 内的“无平方因子”的数有多少个？整数 p 无平方因子，当且仅当不存在 $k > 1$ ，使得 p 是 k^2 的倍数。 $1 \leq n \leq m \leq 10^{12}$ ， $m - n \leq 10^7$ 。

【分析】

对于这样的限制，直接枚举判断会超时：需要判断 10^7 个整数，而每个整数还需要花费一定的时间判断是否没有平方因子。怎么办呢？在介绍具体算法之前，需要学会用 Eratosthenes 筛法构造 $1 \sim n$ 的素数表。

筛法的思想特别简单：对于不超过 n 的每个非负整数 p ，删除 $2p, 3p, 4p, \dots$ ，当处理完所有数之后，还没有被删除的就是素数。如果用 $\text{vis}[i]$ 表示 i 已经被删除，筛法的代码可以写成：

```
memset(vis, 0, sizeof(vis));
for(int i = 2; i <= n; i++)
    for(int j = i*2; j <= n; j+=i) vis[j] = 1;
```

尽管可以继续改进，但这份代码已经相当高效了。为什么呢？给定外层循环变量 i ，内层循环的次数是 $\left\lfloor \frac{n}{i} \right\rfloor - 1 < \frac{n}{i}$ 。这样，循环的总次数小于 $\frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{n} = O(n \log n)$ 。这个结论来源于欧拉在 1734 年得到的结果： $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} = \ln(n+1) + \gamma$ ，其中欧拉常数 $\gamma \approx 0.577218$ 。

这样低的时间复杂度允许在很短的时间内得到 10^6 以内的所有素数。

下面来改进这份代码。首先，在“对于不超过 n 的每个非负整数 p ”中， p 可以限定为素数——只需在第二重循环前加一个判断 $\text{if}(!\text{vis}[i])$ 即可。另外，内层循环也不必从 $i*2$ 开始——它已经在 $i=2$ 时被筛掉了。改进后的代码如下：

```
int m = sqrt(n+0.5);
memset(vis, 0, sizeof(vis));
for(int i = 2; i <= m; i++) if(!vis[i])
    for(int j = i*i; j <= n; j+=i) vis[j] = 1;
```

这里有一个有意思的问题：给定的 n ， c 的值是多少呢？换句话说，不超过 n 的正整数中，有多少个是素数呢？

素数定理： $\pi(x) \sim \frac{x}{\ln x}$ 。

其中， $\pi(x)$ 表示不超过 x 的素数的个数。上述定理的直观含义是：它和 $x/\ln x$ 比较接

近——对于算法入门来说，这已足够。表 10-1 给出了一些值来加深读者的印象。

表 10-1 素数定理的直观验证

N	10^2	10^3	10^4	10^5	10^6	10^7	10^8
$\pi(n)$	25	168	1229	9592	78498	664579	5761455
$n/\ln n$	22	145	1086	8686	72382	620421	5428681

最后回到原題：如何求出区间内无平方因子的数？方法和筛素数是类似的：对于不超过 $\sqrt{m}$ 的所有素数 p ，筛掉区间 $[n, m]$ 内 p^2 的所有倍数。

10.1.3 扩展欧几里德算法

直线上的点。求直线 $ax+by+c=0$ 上有多少个整点 (x,y) 满足 $x \in [x_1, x_2]$, $y \in [y_1, y_2]$ 。

【分析】

在解决这个问题之前，首先学习扩展欧几里德算法——找出一对整数 (x,y) ，使得 $ax+by=\gcd(a,b)$ 。注意，这里的 x 和 y 不一定是正数，也可能是负数或者 0。例如， $\gcd(6,15)=3$ ， $6*3-15*1=3$ ，其中 $x=3$ ， $y=-1$ 。这个方程还有其他解，如 $x=-2$ ， $y=1$ 。

下面是扩展欧几里德算法的程序：

```
void gcd(int a, int b, int& d, int& x, int& y) {
    if(!b){ d = a; x = 1; y = 0; }
    else{ gcd(b, a%b, d, y, x); y -= x*(a/b); }
}
```

用数学归纳法并不难证明算法的正确性，此处略去。注意在递归调用时， x 和 y 的顺序变了，而边界也是不难得出的： $\gcd(a,0)=1*a-0*0=a$ 。这样，唯一需要记忆的是 $y=-x*(a/b)$ ，哪怕暂时不懂得其中的原因也不要紧。

上面求出了 $ax+by=\gcd(a,b)$ 的一组解 (x_1,y_1) ，那么其他解呢？任取另外一组解 (x_2,y_2) ，则 $ax_1+by_1=ax_2+by_2$ （它们都等于 $\gcd(a,b)$ ），变形得 $a(x_1-x_2)=b(y_2-y_1)$ 。假设 $\gcd(a,b)=g$ ，方程左右两边同时除以 g ^①，得 $a'(x_1-x_2)=b'(y_2-y_1)$ ，其中 $a'=a/g$ ， $b'=b/g$ 。注意，此时 a' 和 b' 互素，因此 x_1-x_2 一定是 b' 的整数倍。设它为 kb' ，计算得 $y_2-y_1=ka'$ 。注意，上面的推导过程并没有用到“ $ax+by$ 的右边是什么”，因此得出如下结论。

提示 10-1： 设 a, b, c 为任意整数。若方程 $ax+by=c$ 的一组整数解为 (x_0,y_0) ，则它的任意整数解都可以写成 (x_0+kb', y_0-ka') ，其中 $a'=a/\gcd(a,b)$ ， $b'=b/\gcd(a,b)$ ， k 取任意整数。

有了这个结论，移项得 $ax+by=-c$ ，然后求出一组解即可。例如：

例 1： $6x+15y=9$ 。根据欧几里德算法，已经得到了 $6 \times (-2) + 15 \times 1 = 3$ ，两边同时乘以 3 得 $6 \times (-6) + 15 \times 3 = 9$ ，即 $x=-6$ ， $y=3$ 时 $6x+15y=9$ 。

例 2： $6x+15=8$ ，两边除以 3 得 $2x+5=8/3$ 。左边是整数，右边不是整数，显然无解。综合起来，有下面的结论。

^① 如果 $g=0$ ，意味着 a 或 b 等于 0，可以特殊判断。

提示 10-2: 设 a, b, c 为任意整数, $g=\gcd(a,b)$, 方程 $ax+by=g$ 的一组解是 (x_0, y_0) , 则当 c 是 g 的倍数时 $ax+by=c$ 的一组解是 $(x_0c/g, y_0c/g)$; 当 c 不是 g 的倍数时无整数解。

这样, 即完整地解决了本问题。顺便说一句, 本题的名称为什么叫“直线上的点”呢? 这是因为在平面坐标系下, $ax+by+c=0$ 是一条直线的方程。

10.1.4 同余与模算术

你需要花多少时间做下面这道题目呢?

$123456789 * 987654321 = (\quad)$

A. 121932631112635266 B. 121932631112635267

C. 121932631112635268 D. 121932631112635269

既然是选择题, 不必费力把答案完整地计算出来——4 个选项的个位数都不相同, 因此只需要计算出答案的最后一位即可。不难得出, 它等于 $1*9=9$ 。把刚才的解题过程抽象出来就是下面的式子:

$$123456789 * 987654321 \bmod 10 = ((123456789 \bmod 10) * (987654321 \bmod 10)) \bmod 10$$

其中 $a \bmod b$ 表示 a 除以 b 的余数, C 语言表达式是 $a \% b$ 。在本章中, b 一定是正整数, 尽管 $b < 0$ 时表达式 $a \% b$ 也是合法的 (但 $b = 0$ 时会出现除零错)。

不难得到下面的公式:

$$(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$$

$$(a - b) \bmod n = ((a \bmod n) - (b \bmod n) + n) \bmod n$$

$$ab \bmod n = (a \bmod n)(b \bmod n) \bmod n$$

注意在减法中, 由于 $a \bmod n$ 可能小于 $b \bmod n$, 需要在结果加上 n , 而在乘法中, 需要注意 $a \bmod n$ 和 $b \bmod n$ 相乘是否会溢出。例如, 当 $n=10^9$ 时, $ab \bmod n$ 一定在 int 范围内, 但 $a \bmod n$ 和 $b \bmod n$ 的乘积可能会超过 int 。需要用 long long 保存中间结果, 例如:

```
int mul_mod(int a, int b, int n) {
    a %= n; b %= n;
    return (int)((long long)a * b % n);
}
```

当然, 如果 n 本身超过 int 但又在 long long 范围内, 上述方法就不适用了。在这种情况下, 建议初学者使用高精度乘法——尽管有办法可以避免, 但技巧性很强, 不推荐初学者学习。

大整数取模。输入正整数 n 和 m , 输出 $n \bmod m$ 的值。 $n \leq 10^{100}$, $m \leq 10^9$ 。

【分析】

首先, 把大整数写成“自左向右”的形式: $1234 = ((1*10+2)*10+3)*10+4$, 然后用前面的公式, 每步取模, 例如:

```
scanf("%s%d", n, &m);
int len = strlen(n);
int ans = 0;
```

```
for(int i = 0; i < len; i++)
    ans = (int)((long long)ans*10 + n[i] - '0') % m;
printf("%d\n", ans);
```

当然，也可以把 `ans` 声明成 `long long` 类型的，然后在输出时临时转换为 `int`，但要注意乘法溢出的问题。

幂取模。输入正整数 a 、 n 和 m ，输出 $a^n \bmod m$ 的值。 $a, n, m \leq 10^9$ 。

【分析】

很容易写出下面的代码：

```
int pow_mod(int a, int n, int m) {
    int ans = 1;
    for(int i = 0; i < n; i++) ans = (int)((long long)ans * a % m);
}
```

这个函数的时间复杂度为 $O(n)$ ，当 n 很大时速度很不理想。有没有办法算得更快呢？可以利用分治法：

```
int pow_mod(int a, int n, int m) {
    if(n == 0) return 1;
    int x = pow_mod(a, n/2, m);
    long long ans = (long long)x * x % m;
    if (n%2 == 1) ans = ans * a % m;
    return (int)ans;
}
```

例如， $a^{29} = (a^{14})^2 * a$ ，而 $a^{14} = (a^7)^2$ ， $a^7 = (a^3)^2 * a$ ， $a^3 = a^2 * a$ ，一共只做了 7 次乘法。不知读者有没有发现，上述递归方式和二分查找很类似——每次规模近似减小一半。因此，时间复杂度为 $O(\log n)$ ，比 $O(n)$ 好了很多。

模线性方程组。输入正整数 a, b, n ，解方程 $ax \equiv b \pmod{n}$ 。 $a, b, n \leq 10^9$ 。

【分析】

本题中出现了一个新记号：同余。 $a \equiv b \pmod{n}$ 的含义是“ a 和 b 关于模 n 同余”，即 $a \bmod n = b \bmod n$ 。不难得出， $a \equiv b \pmod{n}$ 的充要条件是： $a-b$ 是 n 的整数倍。

提示 10-3： $a \equiv b \pmod{n}$ 的含义是“ a 和 b 除以 n 的余数相同”，其充要条件是“ $a-b$ 是 n 的整数倍”。

这样，原来的方程就可以理解成： $ax-b$ 是 n 的正整数倍。设这个“倍数”为 y ，则 $ax-b=ny$ ，移项得 $ax-ny=b$ ，这恰好就是 10.1.3 节介绍的不定方程（ a, n, b 是已知量， x 和 y 是未知数）！接下来的步骤不再介绍。唯一需要说明的是，如果 x 是方程的解，满足 $x \equiv y \pmod{n}$ 的其他整数 y 也是方程的解。因此，当谈到同余方程的一个解时，其实指的是一个同余等价类。

尽管算法已无须继续讨论，有一个特殊情况需要引起读者重视。 $b=1$ 时， $ax \equiv 1 \pmod{n}$ 的解称为 a 关于模 n 的逆（inverse），它类似于实数运算中“倒数”的概念。什么时候 a 的逆存在呢？根据上面的讨论，方程 $ax-ny=1$ 要有解。这样，1 必须是 $\gcd(a, n)$ 的倍数，因

此 a 和 n 必须互素 (即 $\gcd(a, n) = 1$)。在满足这个条件的前提下, $ax \equiv 1 \pmod{n}$ 只有唯一解。注意, 同余方程的解是指一个等价类。

提示 10-4: 方程 $ax \equiv 1 \pmod{n}$ 的解称为 a 关于模 n 的逆。当 $\gcd(a, n) = 1$ 时, 该方程有唯一解; 否则, 该方程无解。

10.1.5 应用举例

例题 10-1 巨大的斐波那契数! (Colossal Fibonacci Numbers!, UVa11582)

输入两个非负整数 a 、 b 和正整数 n ($0 \leq a, b < 2^{64}$, $1 \leq n \leq 1000$), 你的任务是计算 $f(a^b)$ 除以 n 的余数。其中 $f(0) = f(1) = 1$, 且对于所有非负整数 i , $f(i+2) = f(i+1) + f(i)$ 。

【分析】

所有计算都是对 n 取模的, 不妨设 $F(i) = f(i) \bmod n$ 。不难发现, 当二元组 $(F(i), F(i+1))$ 出现重复时, 整个序列就开始重复。例如, $n=3$, 序列 $F(i)$ 的前 10 项为 1, 1, 2, 0, 2, 2, 1, 0, 1, 1, 第 9、10 项和前两项完全一样。根据递推公式, 第 11 项会等于第 3 项, 第 12 项等于第 4 项……

多久会出现重复呢? 因为余数最多 n 种, 所以最多 n^2 项就会出现重复。设周期为 M , 则只需计算出 $F(0) \sim F(n^2)$, 然后算出 $F(a^b)$ 等于其中的哪一项即可。

例题 10-2 不爽的裁判 (Disgruntled Judge, NWERC 2008, UVa12169)

有个裁判出的题太难, 总是没人做, 所以他很不爽。有一次他终于忍不住了, 心想: “反正我的题没人做, 我干嘛要费那么多心思出题? 不如就输入一个随机数, 输出一个随机数吧。”

于是他找了 3 个整数 x_1 、 a 和 b , 然后按照递推公式 $x_i = (ax_{i-1} + b) \bmod 10001$ 计算出了一个长度为 $2T$ 的数列, 其中 T 是测试数据的组数。然后, 他把 T 和 $x_1, x_3, \dots, x_{2T-1}$ 写到输入文件中, $x_2, x_4, \dots, x_{2T}$ 写到了输出文件中。

你的任务就是解决这个疯狂的题目: 输入 $T, x_1, x_3, \dots, x_{2T-1}$, 输出 $x_2, x_4, \dots, x_{2T}$ 。输入保证 $T \leq 100$, 且输入的所有 x 值为 $0 \sim 10000$ 的整数。如果有多种可能的输出, 任意输出一个即可。

【分析】

如果知道了 a , 就可以计算出 x_2 , 进而根据 $x_3 = (ax_2 + b) \bmod 10001$ 算出 b 。有了 x_1 、 a 和 b , 就可以在 $O(T)$ 时间内计算出整个序列了。如果在计算过程中发现和输入矛盾, 则这个 a 是非法的。由于 a 是 $0 \sim 10000$ 的整数 (因为递推公式对 10001 取模), 即使枚举所有的 a , 时间效率也足够高。

例题 10-3 选择与除法 (Choose and Divide, UVa10375)

已知 $C(m, n) = m! / (n!(m-n)!)$, 输入整数 p, q, r, s ($p \geq q, r \geq s, p, q, r, s \leq 10000$), 计算 $C(p, q) / C(r, s)$ 。输出保证不超过 10^8 , 保留 5 位小数。

【分析】

本题正是唯一分解定理的用武之地。组合数 $C(m, n)$ 的性质将在 10.2.1 节中介绍, 本题只需要用到它的定义。

首先, 求出 10000 以内的所有素数 `primes`, 然后用数组 `e` 表示当前结果的唯一分解式中各个素数的指数。例如, $e=\{1,0,2,0,0,0,\dots\}$ 表示 $2^1 \cdot 5^2 = 50$ 。主程序如下:

```
while(cin >> p >> q >> r >> s) {
    memset(e, 0, sizeof(e));
    add_factorial(p, 1);
    add_factorial(q, -1);
    add_factorial(p-q, -1);
    add_factorial(r, -1);
    add_factorial(s, 1);
    add_factorial(r-s, 1);
    double ans = 1;
    for(int i = 0; i < primes.size(); i++)
        ans *= pow(primes[i], e[i]);
    printf("%.5lf\n", ans);
}
```

其中 `add_factorial(n,d)` 表示把结果乘以 $(n!)^d$, 它的实现如下:

//乘以或除以 n . $d=0$ 表示乘, $d=-1$ 表示除

```
void add_integer(int n, int d) {
    for(int i = 0; i < primes.size(); i++) {
        while(n % primes[i] == 0) {
            n /= primes[i];
            e[i] += d;
        }
        if(n == 1) break; //提前终止循环, 节约时间
    }
}
```

```
void add_factorial(int n, int d) {
    for(int i = 1; i <= n; i++)
        add_integer(i, d);
}
```

例题 10-4 最小公倍数的最小和 (Minimum Sum LCM, UVa10791)

输入整数 n ($1 \leq n < 2^{31}$), 求至少两个正整数, 使得它们的最小公倍数为 n , 且这些整数的和最小。输出最小的和。

【分析】

本题再次用到了唯一分解定理。设唯一分解式 $n = a_1^{p_1} \cdot a_2^{p_2} \cdots$, 不难发现每个 $a_i^{p_i}$ 作为一个单独的整数时最优。

如果就这样匆匆编写程序, 可能会掉入陷阱。本题有好几个特殊情况要处理: $n=1$ 时答案为 $1+1=2$; n 只有一种因子时需要加个 1, 还要注意 $n=2^{31}-1$ 时不要溢出。